# Core Agent Flow

## 1. Understand the intent

The agent should first classify the user's goal:

**Fraud detection**
Detects rings, shared devices, suspicious money movement, mule accounts, repeated behavior, anomaly paths.

**Entity resolution**
Resolve whether records represent the same person, business, account, device, address, or household.

**Customer 360**
Connect customers, accounts, products, interactions, transactions, cases, and preferences.

**Recommendation / similarity**
Find similar users, products, behaviors, or entities.

The agent should ask or infer:

- What are the key entities?
- What decisions need to be made?
- What questions or queries must the graph answer?
- What relationships matter?
- Is the graph operational, analytical, AI/RAG, or ML-focused?

## 2. Profile the tabular data

The agent should inspect each table and column to identify:

- Primary keys
- Foreign keys
- Repeated identifiers
- Candidate entities
- Relationship columns
- Timestamps
- Amounts / numeric measures
- Categorical attributes
- PII or identity attributes
- High-cardinality columns
- Columns that look like devices, IPs, emails, phones, addresses, accounts, merchants, transactions, documents, etc.

Example:

| Column | Likely Meaning |
|---|---|
| customer_id | Vertex ID |
| account_id | Vertex ID |
| device_id | Shared entity |
| transaction_id | Event vertex |
| merchant_id | Counterparty vertex |
| amount | Edge or transaction attribute |
| timestamp | Edge/event attribute |
| email / phone / address | Identity resolution attribute or entity vertex |

## 3. Convert use case into graph questions

Before generating schema, the agent should create target queries.

For fraud detection:

- Which accounts share the same device, phone, email, or IP?
- Are multiple accounts transacting with the same merchant or beneficiary?
- Are there circular payment patterns?
- Are there paths between known fraud accounts and new accounts?
- Which communities show abnormal transaction density?

For entity resolution:

- Which records share strong identifiers?
- Which entities match across weak identifiers?
- What is the confidence score for a resolved entity?
- Which records belong to the same household, business, or identity cluster?

This is critical because **schema should be optimized for queries**, not just mirror the tables.

## 4. Decide what becomes a vertex

A table or column should become a vertex when it represents a durable object that users may want to traverse, score, resolve, or reuse.

Typical vertices:

**Fraud**

- Customer
- Account
- Transaction
- Merchant
- Device
- IPAddress
- Phone
- Email
- Address
- Card
- Beneficiary
- FraudCase

**Entity resolution**

- PersonRecord
- ResolvedEntity
- Email
- Phone
- Address
- Document
- Business
- Identifier

Rule of thumb:

> If the value connects many records together, make it a vertex.

Example: `device_id` should usually be a `Device` vertex, not just an Account attribute, because shared devices are fraud signals.

## 5. Decide what becomes an edge

Edges should represent meaningful relationships that support traversal.

Examples:

- Customer — owns — Account
- Account — made — Transaction
- Transaction — paid — Merchant
- Account — used — Device
- Customer — has — Email
- Customer — has — Phone
- PersonRecord — matches — ResolvedEntity
- Account — transferred_to — Account
- Transaction — originated_from — IPAddress

Edges should include attributes when the relationship has context:

- timestamp
- amount
- channel
- confidence score
- match reason
- source system
- status
- direction
- risk score

## 6. Choose directionality

The agent should define edge direction based on query patterns.

Examples:

- `Account -> Transaction`
- `Transaction -> Merchant`
- `Account -> Device`
- `PersonRecord -> Email`
- `PersonRecord -> Phone`
- `Account -> Account` for transfers

For TigerGraph, the agent should also consider reverse traversals. If queries commonly go both ways, the schema should include reverse edges or use undirected-style modeling where appropriate.

## 7. Preserve events when needed

The agent should decide whether something is:

- an entity
- a relationship
- an event

For fraud, transactions are usually best modeled as **event vertices**, not just edges, because they have rich attributes:

- amount
- timestamp
- channel
- location
- merchant
- status
- risk score
- authorization result

This enables paths like:

```
Account -> Transaction -> Merchant
```

instead of overloading an edge with too much behavior.

## 8. Generate a candidate TigerGraph schema

The agent output should include:

- Vertex types
- Edge types
- Attributes
- Primary IDs
- Reverse edges
- Loading mappings
- Example GSQL queries
- Explanation of why the schema supports the use case

Example output:

```
None
Vertex Types:
Customer(customer_id, name, risk_score)
```

```
Account(account_id, account_type, open_date, status)
Transaction(transaction_id, amount, timestamp, channel)
Device(device_id, device_type)
Merchant(merchant_id, merchant_category)

Edge Types:
Customer_OWNS_Account(Customer -> Account)
Account_MADE_Transaction(Account -> Transaction)
Transaction_PAID_Merchant(Transaction -> Merchant)
Account_USED_Device(Account -> Device, first_seen, last_seen)
```

### 9. Validate schema against target queries

The agent should simulate whether the schema can answer the required questions.

For each use case query, it should check:

- Are all required entities present?
- Are relationships traversable?
- Are timestamps available?
- Are attributes on the right object?
- Are high-value joins converted into graph edges?
- Are important many-to-many relationships modeled as vertices or edges?
- Would the traversal be efficient?

If not, revise the schema.

### 10. Score and explain the schema

The agent should produce a schema quality score based on:

- Use-case fit
- Query coverage
- Traversal efficiency
- Data completeness
- Entity reuse
- Attribute placement
- Graph algorithm readiness

- ML / GraphRAG readiness
- Simplicity

Example:

```
None
Schema Quality Score: 87/100

Strengths:
- Shared device, phone, and IP relationships are modeled as
vertices.
- Transaction events are preserved as vertices.
- Schema supports multi-hop fraud ring detection.

Gaps:
- No address normalization field found.
- Merchant hierarchy is missing.
- No known fraud label available for supervised ML.
```

# Recommended Agent Architecture

## Planner Agent

Understands use case, extracts goals, defines target questions.

## Data Profiler Agent

Analyzes tables, columns, keys, cardinality, nulls, and candidate relationships.

## Schema Designer Agent

Creates the graph model: vertices, edges, attributes, directionality.

## Query Validator Agent

Generates representative GSQL queries and checks whether the schema supports them.

## Critic Agent

Reviews the schema for missing relationships, over-modeling, bad directionality, or weak use-case fit.

### TigerGraph Generator Agent

Produces TigerGraph outputs:

- schema definition
- loading job mapping
- starter GSQL queries
- sample algorithms
- documentation summary

## Simple logic pattern

```
None
User Intent + Source Tables
        ↓
Infer Use Case
        ↓
Generate Target Questions
        ↓
Profile Tables and Columns
        ↓
Identify Entities, Events, Identifiers, Measures
        ↓
Map Entities to Vertices
        ↓
Map Relationships to Edges
        ↓
Place Attributes
        ↓
Validate Against Target Queries
        ↓
Generate TigerGraph Schema + Loading Jobs + Queries
        ↓
Explain, Score, and Iterate
```

Logic for use case should live in the control plane, and only deployed to a workspace on demand

Recommended model:

```
None
Control Plane
 - deterministic logic
 - prompt registry
 - use-case pattern library
 - validation rules
 - versioning
 - governance
 - deployment controls

Workspace
 - enabled agent capability
 - workspace-specific configuration
 - customer data context
 - selected use case
 - generated artifacts
 - accepted schema versions
```

# 1. Deterministic use-case logic

This should live in your **TigerGraph backend / orchestrator service**.

Example:

```
None
Cloud Run / Agent Runtime
 → Fraud schema rules
```

```
→ Data profiling rules
→ Validation rules
→ TigerGraph schema generator
```

This is where you encode known TigerGraph graph-design patterns for fraud:

- `Account`, `Customer`, `Transaction`, `Merchant`, `Device`, `IPAddress`, `Email`, `Phone`, `Address`
- Shared identifiers should become vertices
- Transactions should usually become event vertices
- Money movement should support directional traversal
- Devices, IPs, phones, and emails should support reverse lookup
- Known-fraud labels should be preserved
- Timestamp and amount should be retained for temporal fraud analysis

This logic should be versioned and testable.

## 2. Prompt / reasoning logic

This should live in the **prompt registry / agent configuration**.

Example:

```
None
Fraud Detection Schema Prompt v1.0
```

This gives the LLM instructions such as:

```
None
You are designing a TigerGraph schema for fraud detection.
Prioritize multi-hop traversal, shared identifiers, ring detection,
temporal analysis, and explainability.
Do not simply mirror relational tables.
Convert high-cardinality shared identifiers into vertices when they
create meaningful fraud connectivity.
```

The LLM uses this to reason through ambiguous cases.

## 3. Knowledge / pattern library

This should live in a **retrievable knowledge base**.

Example:

```
None
Fraud schema patterns
Entity resolution schema patterns
TigerGraph best practices
Example GSQL queries
Example loading jobs
Reference architectures
```

For fraud detection, the library would include reusable patterns like:

```
None
Account → Transaction → Merchant
Account → Device
Account → IPAddress
Customer → Email
Customer → Phone
Account → Account transfer
FraudCase → Account
```

The orchestrator retrieves the relevant pattern and injects it into the LLM context.

# Recommended layout

```
None
Savanna UI
 ↓
Natural Language API
 ↓
Agent Orchestrator
 ↓
Use Case Classifier
```

```
 ↓
Fraud Detection Schema Module
     - deterministic rules
     - fraud schema patterns
     - validation checks
 ↓
LLM / Gemini
     - reasoning
     - ambiguity resolution
     - explanation
 ↓
Schema Validator
 ↓
TigerGraph Artifact Generator
 ↓
Human Review
```

## Simple rule

The LLM should **not own the core fraud logic**.

The LLM should help interpret intent and handle ambiguity, but the reusable fraud schema intelligence should be stored in:

```
None
backend rules + prompt templates + knowledge base
```

That gives you:

- repeatability
- version control
- explainability
- easier testing
- better governance
- less hallucination
- reusable patterns across customers

They work together as a **schema generation pipeline**:

```
None
User intent + source metadata
        ↓
Deterministic logic
        ↓
Use case pattern library
        ↓
Prompt / LLM reasoning
        ↓
Validation logic
        ↓
TigerGraph schema proposal
```

# 1. Deterministic logic provides the rules

This is the “hard-coded intelligence.”

Example rules for fraud:

```
None
If column name includes device_id, phone, email, ip_address:
 recommend a shared identifier vertex

If table has transaction_id + amount + timestamp:
 recommend Transaction as an event vertex

If table has source_account_id and target_account_id:
 recommend Account_TO_Account transfer edge

If a column is low-cardinality:
 consider it an attribute, not a vertex
```

The deterministic layer creates the first structured interpretation of the data.

# 2. Pattern library provides reusable fraud designs

This is the “known-good schema knowledge.”

For fraud, the pattern library may say:

```
None
Common fraud schema pattern:

Customer → Account
Account → Transaction
Transaction → Merchant
Account → Device
Account → IPAddress
Customer → Email
Customer → Phone
FraudCase → Account
Account → Account transfer
```

It also provides query patterns:

```
None
Find accounts sharing the same device.
Find accounts within 3 hops of known fraud.
Find circular payment paths.
Find communities with high transaction density.
```

This helps the agent avoid inventing a schema from scratch.

## 3. Prompt / LLM reasoning adapts the design

The prompt tells the LLM how to combine:

- user intent
- source metadata
- deterministic findings
- fraud pattern library
- TigerGraph best practices

The LLM handles ambiguity.

Example:

```
None
Source has customer_id, account_id, device_id, email,
transaction_id, merchant_id.

Deterministic logic says:
- Account, Customer, Transaction, Device, Merchant are candidates.
- device_id and email may be shared identifier vertices.
- Transaction is likely an event vertex.

Pattern library says:
- Fraud requires shared identity relationships and transaction
paths.

LLM decides:
- Make Device and Email vertices, not attributes.
- Model Transaction as a vertex.
- Add Account_USED_Device and Customer_HAS_Email edges.
- Add Account_MADE_Transaction and Transaction_PAID_Merchant edges.
```

## 4. Validator checks the output

The validator ensures the proposed schema is valid and useful.

It checks:

```
None
Are all edge endpoints valid vertices?
Does every vertex have a primary ID?
Can the target fraud queries be answered?
Are shared identifiers modeled as traversable vertices?
Are transactions preserved for temporal analysis?
Is the schema too complex for the POC?
```

## Example end-to-end flow

Input:

```
None
Use case: fraud detection

Tables:
customers(customer_id, name, email, phone)
accounts(account_id, customer_id, open_date, status)
transactions(transaction_id, account_id, merchant_id, amount,
timestamp)
devices(account_id, device_id, ip_address, login_time)
```

Deterministic logic detects:

```
None
customer_id → Customer
account_id → Account
transaction_id → Transaction event
merchant_id → Merchant
device_id → Device shared identifier
ip_address → IPAddress shared identifier
email → Email shared identifier
phone → Phone shared identifier
```

Pattern library adds fraud best practices:

```
None
Use shared identifiers as vertices.
Preserve transactions as event vertices.
Support account-to-merchant and account-to-device traversal.
Support multi-hop fraud ring detection.
```

Prompt / LLM reasoning produces:

```
None
Vertices:
Customer, Account, Transaction, Merchant, Device, IPAddress, Email,
Phone
```

```
Edges:
Customer_OWNS_Account
Account_MADE_Transaction
Transaction_PAID_Merchant
Account_USED_Device
Device_SEEN_AT_IPAddress
Customer_HAS_Email
Customer_HAS_Phone
```

Validator confirms:

```
None
Can answer:
- Which accounts share a device?
- Which customers share a phone/email?
- Which accounts paid the same merchant?
- Which accounts are connected to known fraud entities within 3
hops?
```